

What is a vector database?

A vector database stores data as numerical vectors called embeddings and retrieves the most similar vectors using similarity search.

It is mainly used for semantic search, RAG, recommendation systems, and image search, where results are found by meaning rather than exact keyword matching.

What is vector index?

A vector index is a data structure that organizes embeddings so the vector database can find similar vectors quickly without comparing the query with every stored vector.

Examples include Flat, IVF, and HNSW.

Vector database stores vectors; the vector index makes searching those vectors faster.

What is a similarity search?

Similarity search is the process of finding stored vectors that are closest or most similar to a query vector.

It compares vectors using metrics such as cosine similarity, Euclidean distance, or dot product.

Example: A search for "How can I recover my account?" may retrieve "Steps to reset your password" because both have similar meaning, even though the words are different.

Example of vector database(most used vector database across the industry solution)

FAISS

Chroma DB

Pinecone

Milvus

Qdrant

PostgreSQL + pgvector

OpenSearch/ AWS Open Search

Azure AI Search

In-memory- FAISS, Chroma DB

Local persistent(Local server)- FAISS, Chroma, Qdrant, Milvus

Vendor-managed SaaS(Vector DB)- Pinecone, Qdrant, Milvus, Weaviate

Cloud-provider managed(AWS, Azure, Google) - Azure AI Search , AWS Open search, Vertex AI vector search, BigQuery vector search

Self-hosted

→ You manage everything.

Vendor-managed SaaS

→ The database vendor manages it.

Cloud-provider managed

→ AWS, Azure or GCP manages it.

vector similarity search core types:

Exact Nearest Neighbor Search

Approximate Nearest Neighbor Search (ANN)

Exact search checks the query against every stored vector and returns the true nearest matches.

ANN (Approximate Nearest Neighbor) searches only a selected part of the index to return very close matches faster, but it may occasionally miss the exact best result.

Method Accuracy Speed Best for

Exact Search 100% exact Slower on large datasets Small datasets, evaluation

ANN Search Approximate/high recall Much faster Large-scale production search

← IVF | HNSW

Examples:

Exact search → Flat index

ANN search → HNSW, IVF

Exact search prioritizes accuracy; ANN prioritizes speed and scalability.

Examples:

Pinecone

vector store

SQL: PostgreSQL, MySQL, Oracle

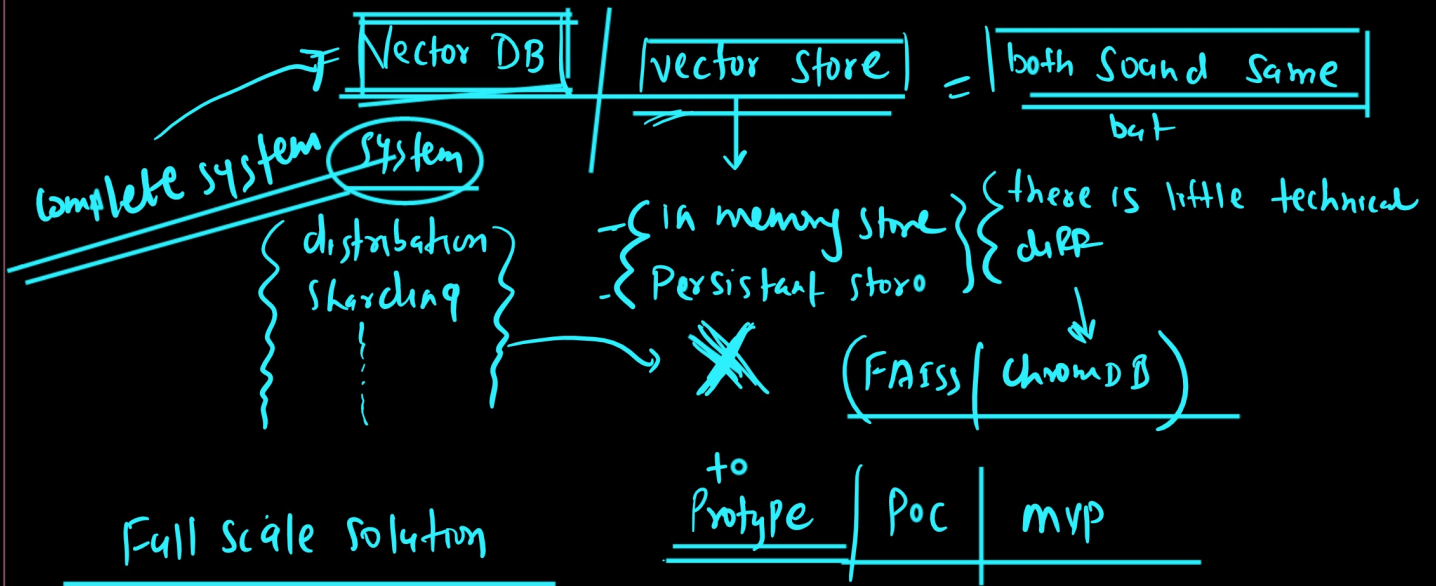
NoSQL: MongoDB, DynamoDB, Cassandra

(S)

Vector DB: Pinecone, Qdrant, Milvus, Weaviate

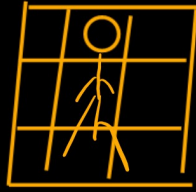
Dedicated Vector Database / Vector Store

Feature	SQL Database	NoSQL Database	Vector Database
Primary purpose	Structured relational data and transactions	Flexible and distributed application data	Embedding storage and similarity retrieval
Data structure	Tables, rows, and columns	Documents, key-value pairs, graphs, or wide columns	Vectors, IDs, original content, and metadata
Schema	Usually predefined and structured	Flexible or schema-light	Fixed vector dimensions with flexible metadata
Main query type	Exact matching, joins, ranges, and aggregations	Key lookup, document queries, and filtering	Semantic similarity and nearest-neighbor search
Typical use cases	Banking, payroll, orders, and inventory	User profiles, sessions, logs, and content catalogs	RAG, recommendations, semantic search, and image search



TEXT → my name is sunny.

IMAGE →

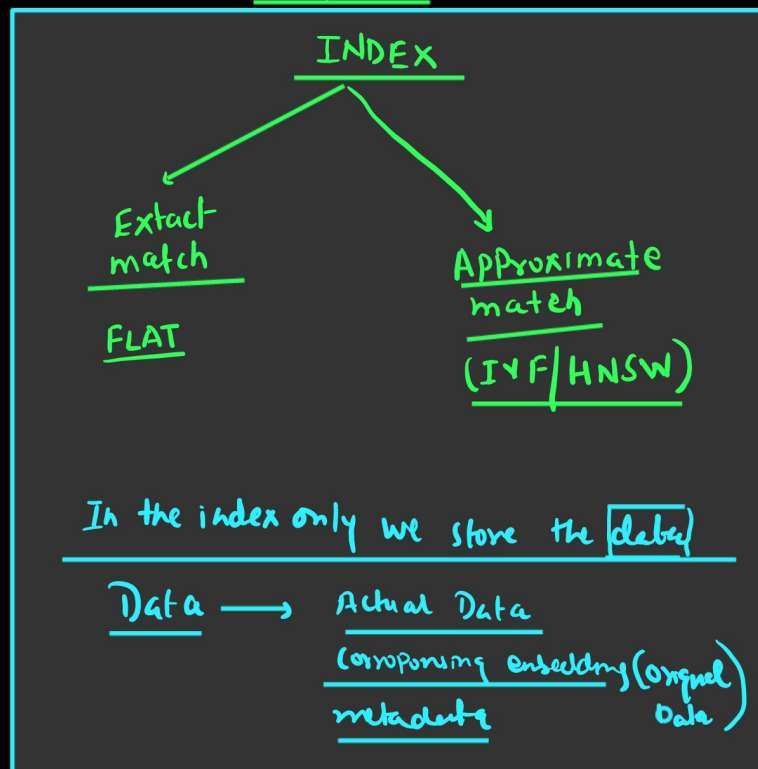


Audio →

Video →

embedding → vector / numeric
= [| | | |]

Vector DB



clustering

Inverted File Index

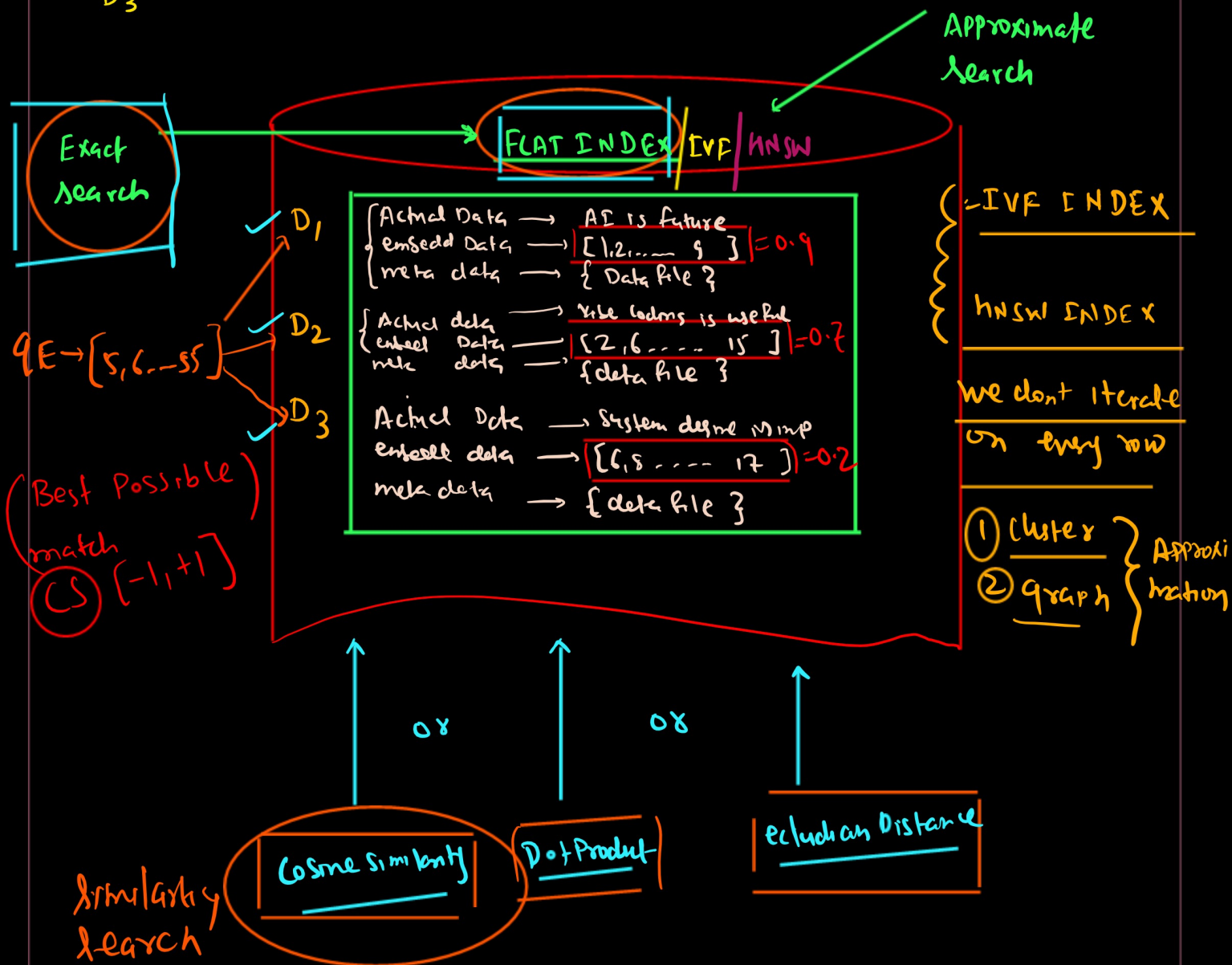
Hierarchical navigable
small world

graph

Data $\rightarrow$ $\left\{ \begin{array}{l} 1. AI \text{ is future.} \\ 2. Vibe coding is useful. \\ 3. System design is imp \end{array} \right\}$

Convert it into embedding

$D_1 \rightarrow [1, 2, 3, 4, 5, 6, 7, 8, 9]$
 $D_2 \rightarrow [2, 6, 8, 9, 10, 13, 14, 15]$
 $D_3 \rightarrow [6, 8, 9, 10, 12, 13, 14, 17]$



Query ML is future $\rightarrow [5, 6, 7, 8, 9, 10, 60, 58]$

$D_1 \rightarrow \left\{ \begin{array}{l} \text{Actual Data} \\ \text{embedd data} \\ \text{meta data} \end{array} \right\} \rightarrow \text{LLM}$

Context

1 million = 10 Lakh

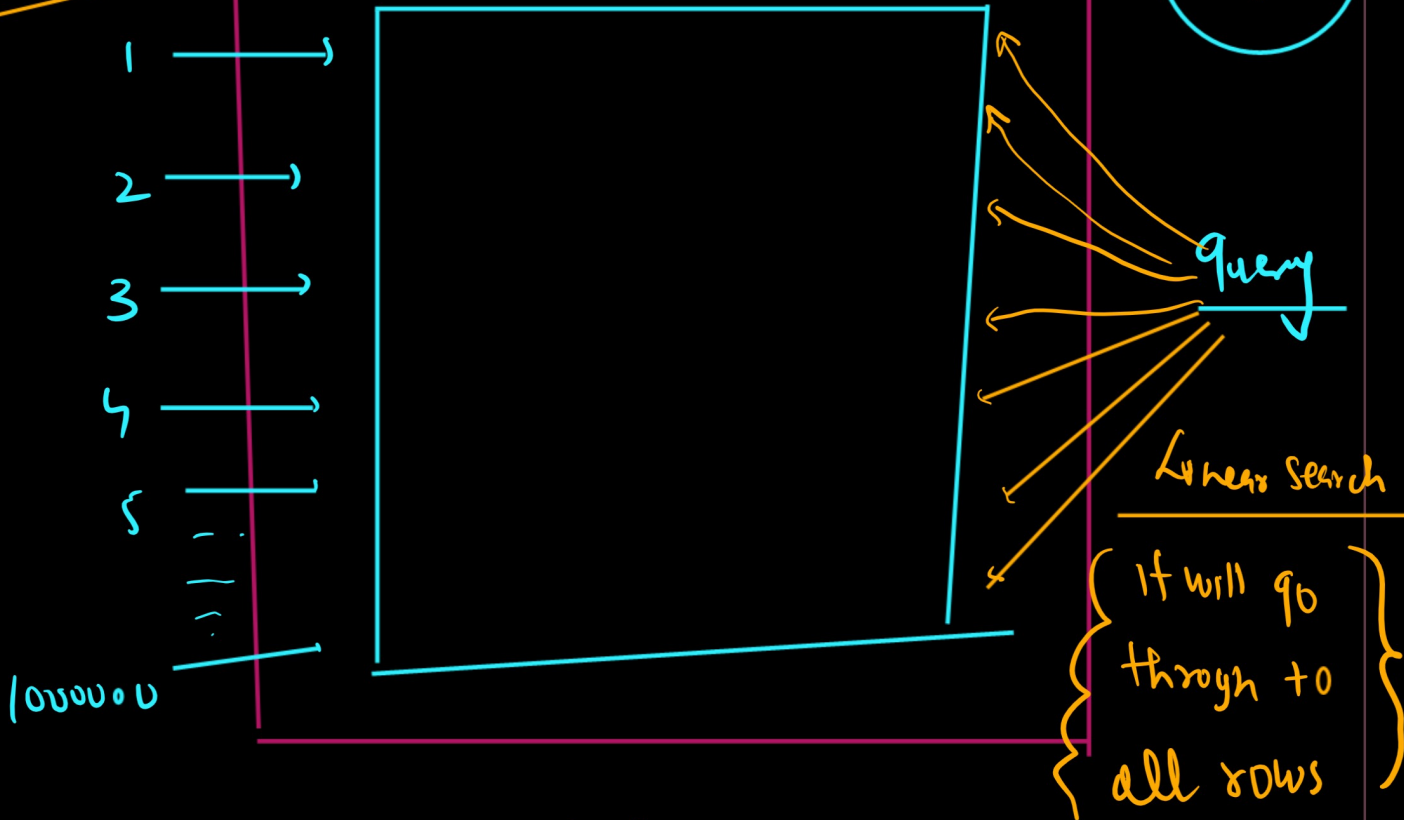
Vector Database

Huge data

IVF / HNSW
INDEX

FLAT INDEX

Exact match



clustering

IVF - INDEX

HNSW - INDEX

Approximate Search

Non-linear Search

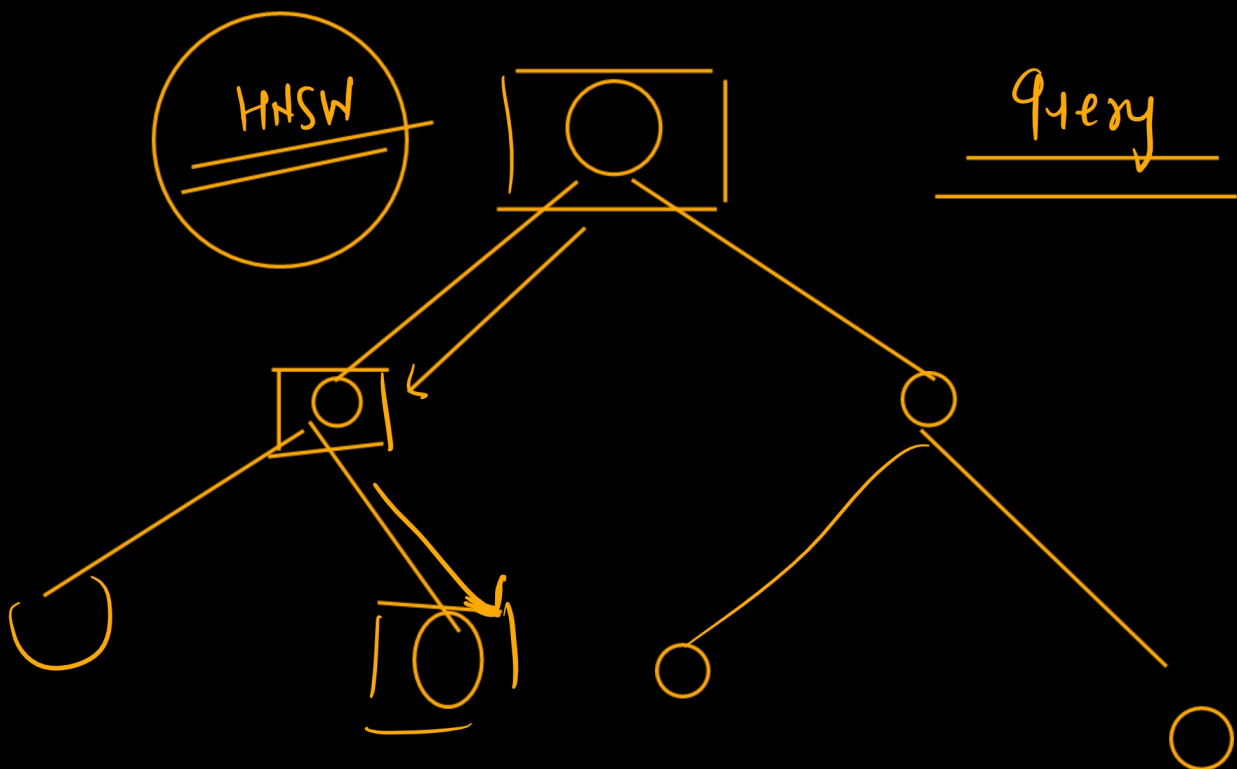
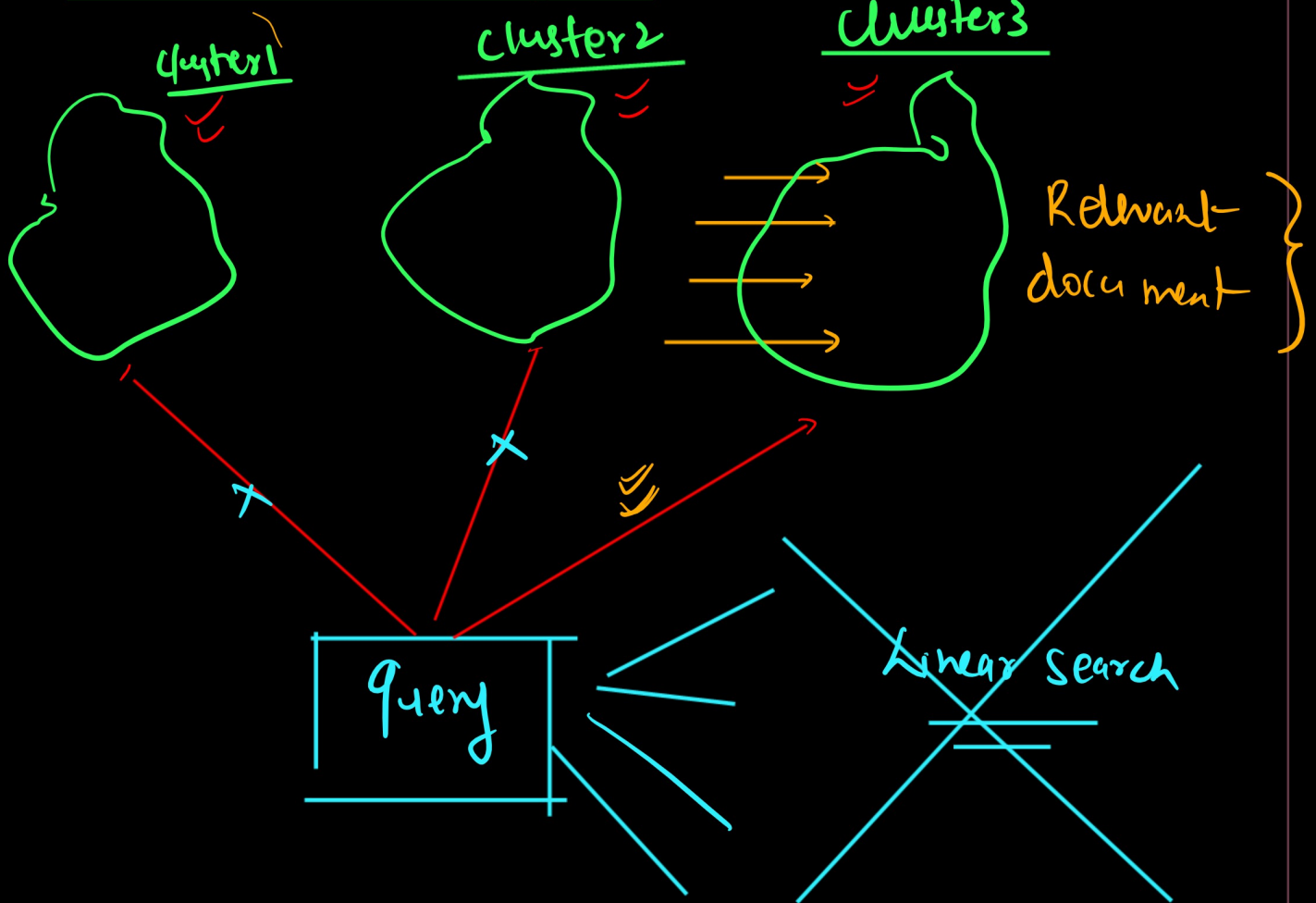
graph

it is a Time consuming

10-million Rows

IVF INDEX

Arrangement of the data



IVP

All vector $\xrightarrow[\text{grouping}]{\text{cluster}}$

{ similar kind of data would be available in one cluster }

- ✓ cluster 1 ← AI related data
- ✗ cluster 2 ← RL related data
- ✓ cluster 3 ← DE " "
- !
- cluster N

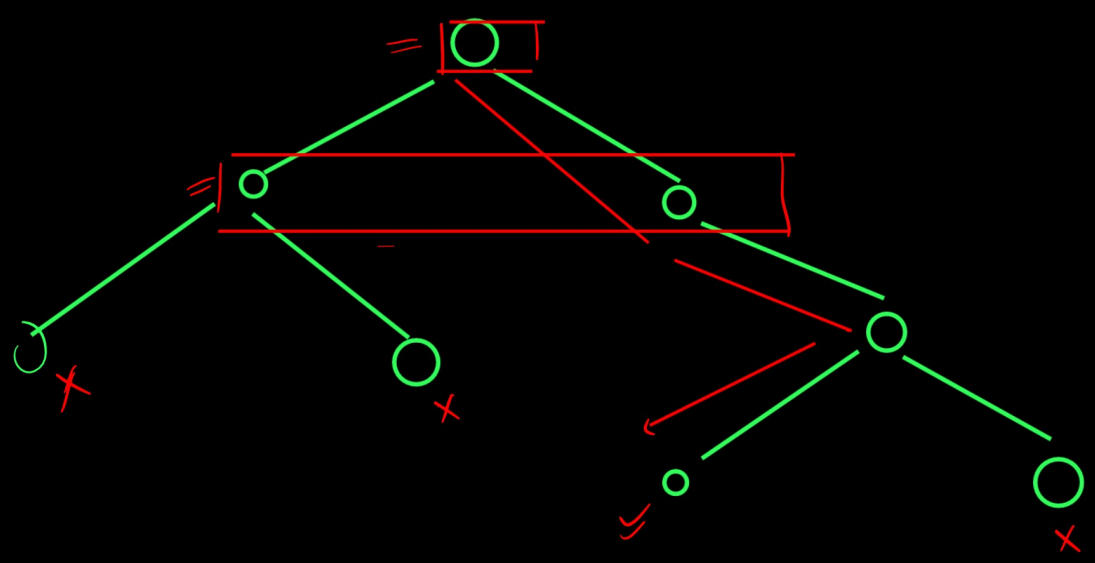
Query $\xrightarrow{\text{embedding (vector)}}$ it will try to find Nearest cluster

AI

Q&A Search space is reducing
FASTER Result

{ Select top k result from this cluster only
(TOP k)
5/10/15

HNSW $\rightarrow$ Hierarchical Navigable small world



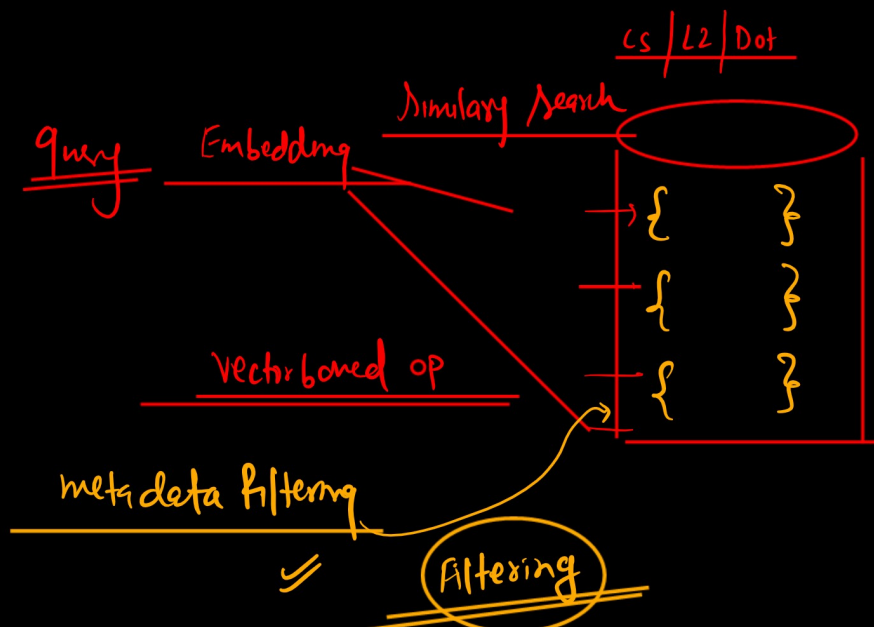
Query $\rightarrow$ Find approximation over top layer

What's stored inside a Vector Database?

ID
Vector/Embedding
Original text or content
Metadata

Example:

```
{  
  "id": "chunk_101",  
  "vector": [0.12, -0.45, 0.88],  
  "text": "Employees receive 20 annual leaves.",  
  "metadata": {  
    "source": "policy.pdf",  
    "page": 12,  
    "department": "HR"  
  }  
}
```



Metadata Filtering

Similarity search can be combined with structured filters.

Example:

Find semantically similar documents
where department = "HR"
and year = 2026

This prevents irrelevant documents from other departments from being retrieved.

CRUD Operations in Vector DB

Students should know:

Create → Insert vectors
Read → Search/retrieve vectors
Update → Replace embedding or metadata
Delete → Remove vectors

CRUD

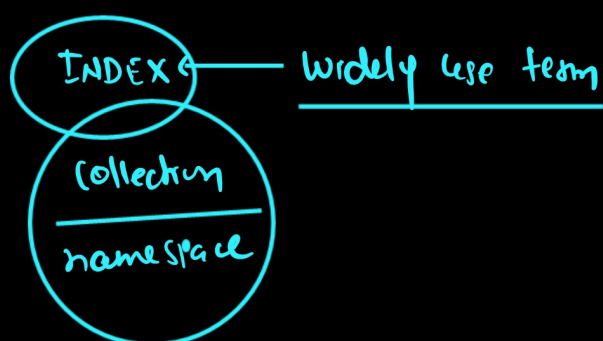
Upsert = Insert if absent, otherwise update

Collections, Indexes and Namespaces

Different databases use different names:

Qdrant → Collection
Pinecone → Index + Namespace
Chroma → Collection
Milvus → Collection
OpenSearch → Index

Concept would be same



Vector Dimension

Every embedding model produces a fixed-size vector.

Example:

Model A → 384 dimensions

Model B → 768 dimensions

Model C → 1536 dimensions

Embed

INDEX → 768

Important:

The vector database collection/index dimension must match the embedding model output dimension.

A 1536-dimension index cannot directly accept a 768-dimension vector.

Vector Database Evaluation

Important evaluation metrics:

Recall@k

Precision@k

Latency

Throughput

Memory usage

Index build time

Storage cost

Is Chunking Always Required?

No. Chunking is not always mandatory.

Chunking is useful when a document or content unit is too large to be retrieved and processed effectively.

Large document

- smaller meaningful retrieval units
- better retrieval
- less irrelevant context
- lower token cost

You can skip chunking when:

Each document is already small.

A database record already contains complete information.

A page or section is already a meaningful retrieval unit.

The complete document can safely fit within the model's context window.

The required information can be retrieved directly using a structured SQL query.

Example:

One FAQ question + answer

- Already an atomic unit
- Further chunking may not be required

However:

A 200-page policy PDF

- Chunking is generally required